T11 Arbre de Décision : Outils de Recherche

Technique Intermediate

Choisir entre Glob, Grep, Read, Agent et Bash selon le contexte

Les 5 outils de recherche natifs

Claude Code propose 5 outils distincts pour trouver et lire des informations dans un projet. Chacun a un rôle précis, et les mélanger sans stratégie coûte des tokens inutilement. `Glob` et `Grep` sont dans le jeu d'outils par défaut sur Windows uniquement ; sous macOS, Linux et WSL, Claude cherche avec `find` et `grep` via `Bash`, sauf activation (par exemple avec `--tools`).

Outil	Rôle	Coût tokens
`Glob`	Trouver des fichiers par pattern	Faible
`Grep`	Chercher du contenu par regex	Faible
`Read`	Lire un fichier entier ou partiel	Élevé
`Agent`	Exploration large, multi-fichiers	Très élevé
`Bash`	Commandes système, cas non couverts	Variable

Glob : fichiers par pattern

Glob retourne une liste de chemins sans lire leur contenu. Utiliser cet outil pour réduire l'ensemble de fichiers avant d'en lire le contenu.

```
Exemples de patterns :
*.ts              → tous les .ts à la racine
src/**/*.tsx      → tous les .tsx dans src/
**/*.test.ts      → tous les fichiers de test
```

Règle d'usage : utiliser Glob en premier pour réduire le périmètre, puis Grep ou Read sur la liste obtenue.

Grep : contenu par regex

Grep utilise ripgrep, adapté aux recherches de texte exact. Il extrait les lignes correspondantes sans charger tout le contenu.

```
# Chercher une fonction par nom exact
rg "authenticate" --type ts

# Chercher avec contexte (3 lignes autour)
rg "validateJWT" -A 3
```

Utiliser Grep pour les recherches de texte exact. Ne pas déléguer à Agent ou Bash si le motif est connu.

Read : lecture ciblée

Read charge un fichier entier ou une plage de lignes via `offset` et `limit`. Sur les gros fichiers, il tronque automatiquement à 2000 lignes.

```
# Lire les lignes 50 à 120 d'un fichier
Read("auth.service.ts", offset=50, limit=71)
```

Toujours préférer un Read ciblé avec offset/limit plutôt qu'un Read complet sur un fichier de plusieurs centaines de lignes.

Agent : exploration large

L'outil Agent (anciennement nommé Task) spawne un sous-agent avec son propre contexte isolé. C'est la bonne option quand on ne sait pas où chercher, que le périmètre est vaste, ou que l'exploration nécessite plusieurs fichiers en parallèle.

Coût : chaque sous-agent ouvre un nouveau contexte, ce qui représente un overhead significatif. À réserver aux tâches qui le justifient réellement.

Bash : l'adaptateur universel

Bash couvre les cas que les 4 autres outils ne savent pas gérer directement : commandes git, appels système, pipes complexes, outils CLI tiers.

```
# Cas d'usage légitimes pour Bash
git log --oneline -20
find . -name "*.lock" -mtime -1
wc -l src/**/*.ts
```

Arbre de décision

```
Je cherche des fichiers par nom/extension
  → Glob

Je cherche du texte/code dont je connais le pattern
  → Grep

Je veux lire un fichier identifié
  → Read (avec offset/limit si fichier long)

Je ne sais pas où chercher, périmètre large
  → Agent (sous-agent)

Aucun des 4 outils ne suffit
  → Bash
```

Principe fondamental : Glob et Grep avant d'ouvrir des fichiers avec Read. Agent uniquement quand le périmètre est inconnu.